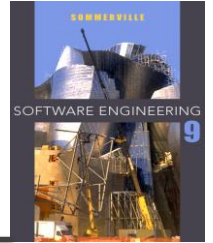


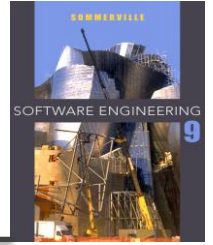
Chapter 9 – Software Evolution

Topics covered



- ✧ Evolution processes
 - Change processes for software systems
- ✧ Program evolution dynamics
 - Understanding software evolution
- ✧ Software maintenance
 - Making changes to operational software systems
- ✧ Legacy system management
 - Making decisions about software change

Software change

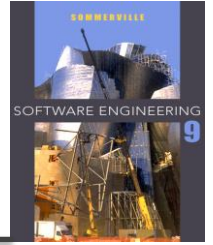


✧ Software change is inevitable

- New requirements emerge when the software is used;
- The business environment changes;
- Errors must be repaired;
- New computers and equipment is added to the system;
- The performance or reliability of the system may have to be improved.

✧ A key problem for all organizations is implementing and managing change to their existing software systems.

Importance of evolution

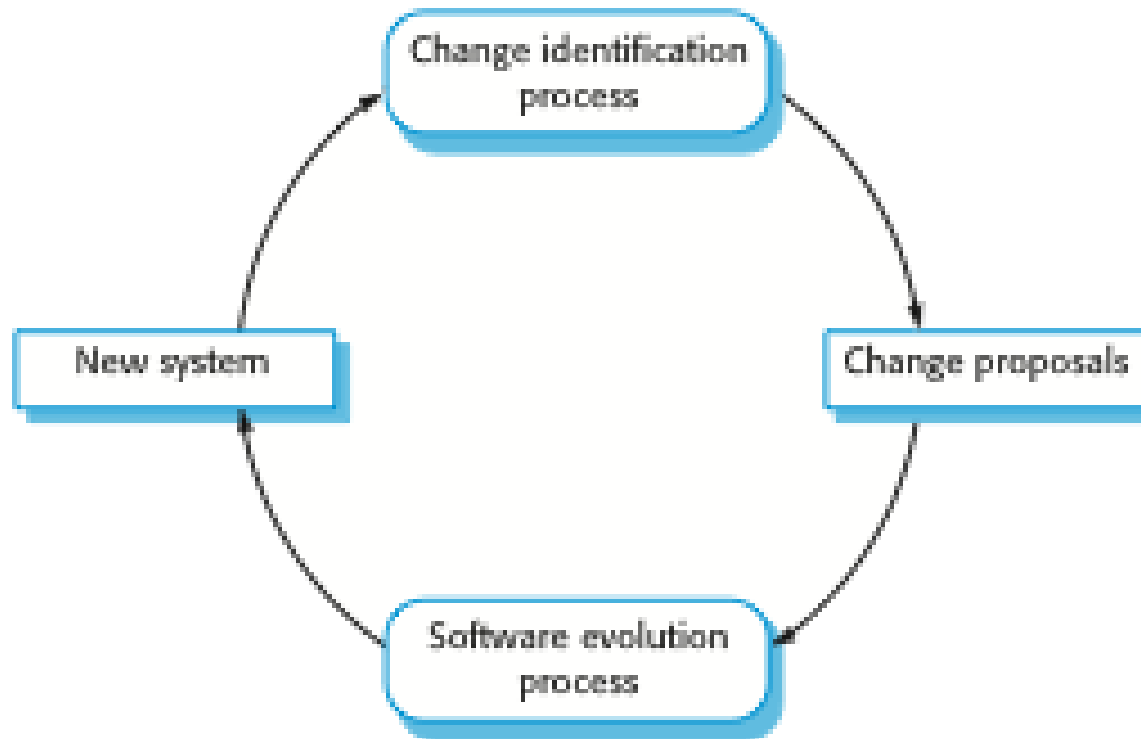
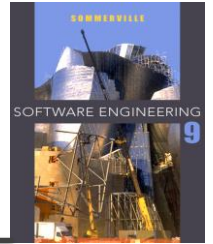


- ✧ Organisations have huge investments in their software systems - they are critical business assets.
- ✧ To maintain the value of these assets to the business, they must be changed and updated.
- ✧ The majority of the software budget in large companies is devoted to changing and evolving existing software rather than developing new software.

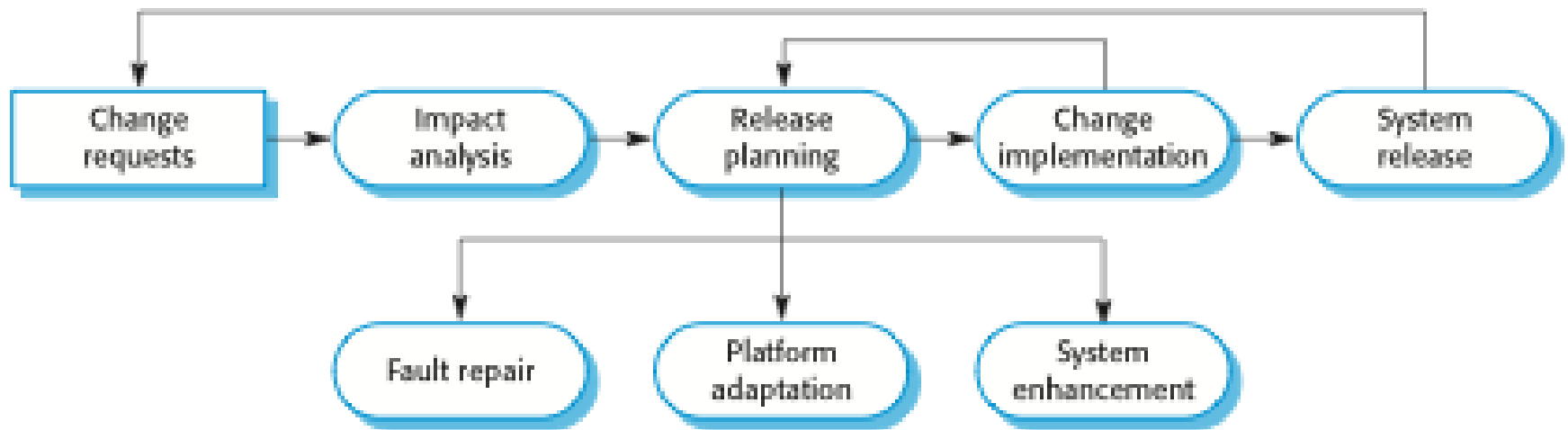
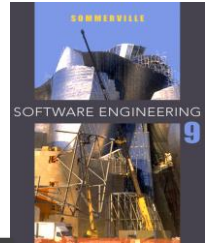
Evolution processes

- ✧ Software evolution processes depend on
 - The type of software being maintained;
 - The development processes used;
 - The skills and experience of the people involved.
- ✧ Proposals for change are the driver for system evolution.
 - Should be linked with components that are affected by the change, thus allowing the cost and impact of the change to be estimated.
- ✧ Change identification and evolution continues throughout the system lifetime.

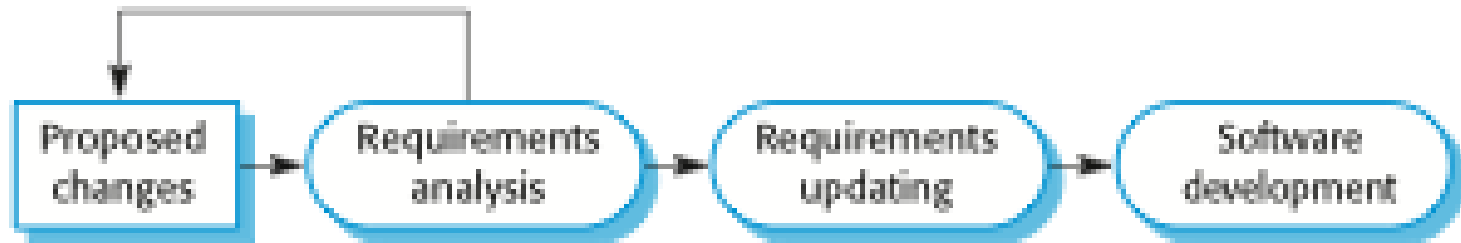
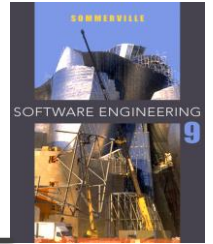
Change identification and evolution processes



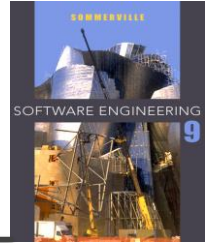
The software evolution process



Change implementation

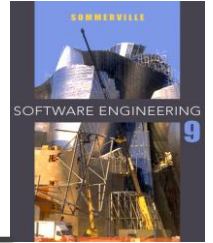


Change implementation



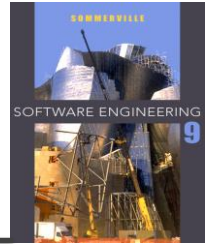
- ✧ Iteration of the development process where the revisions to the system are designed, implemented and tested.
- ✧ A critical difference is that the first stage of change implementation may involve program understanding, especially if the original system developers are not responsible for the change implementation.
- ✧ During the program understanding phase, you have to understand how the program is structured, how it delivers functionality and how the proposed change might affect the program.

Urgent change requests

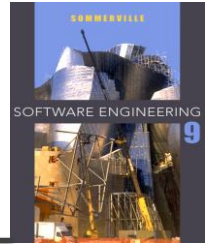


- ✧ Urgent changes may have to be implemented without going through all stages of the software engineering process
 - If a serious system fault has to be repaired to allow normal operation to continue;
 - If changes to the system's environment (e.g. an OS upgrade) have unexpected effects;
 - If there are business changes that require a very rapid response (e.g. the release of a competing product).

The emergency repair process

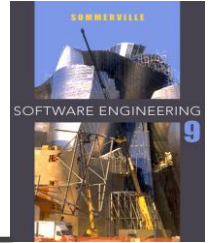


Program evolution dynamics



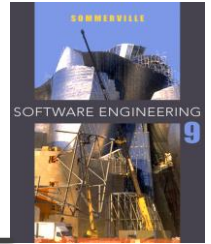
- ✧ *Program evolution dynamics* is the study of the processes of system change.
- ✧ After several major empirical studies, Lehman and Belady proposed that there were a number of 'laws' which applied to all systems as they evolved.
- ✧ There are sensible observations rather than laws. They are applicable to large systems developed by large organisations.
 - It is not clear if these are applicable to other types of software system.

Change is inevitable



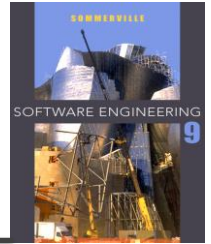
- ✧ The system requirements are likely to change while the system is being developed because the environment is changing. Therefore a delivered system won't meet its requirements!
- ✧ Systems are tightly coupled with their environment. When a system is installed in an environment it changes that environment and therefore changes the system requirements.
- ✧ Systems **MUST** be changed if they are to remain useful in an environment.

Lehman's laws



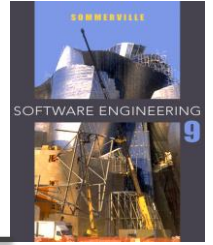
Law	Description
Continuing change	A program that is used in a real-world environment must necessarily change, or else become progressively less useful in that environment.
Increasing complexity	As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving and simplifying the structure.
Large program evolution	Program evolution is a self-regulating process. System attributes such as size, time between releases, and the number of reported errors is approximately invariant for each system release.
Organizational stability	Over a program's lifetime, its rate of development is approximately constant and independent of the resources devoted to system development.

Lehman's laws



Law	Description
Conservation of familiarity	Over the lifetime of a system, the incremental change in each release is approximately constant.
Continuing growth	The functionality offered by systems has to continually increase to maintain user satisfaction.
Declining quality	The quality of systems will decline unless they are modified to reflect changes in their operational environment.
Feedback system	Evolution processes incorporate multiagent, multiloop feedback systems and you have to treat them as feedback systems to achieve significant product improvement.

Software maintenance



- ✧ Modifying a program after it has been put into use.
- ✧ The term is mostly used for changing custom software. Generic software products are said to evolve to create new versions.
- ✧ Maintenance does not normally involve major changes to the system's architecture.
- ✧ Changes are implemented by modifying existing components and adding new components to the system.

Types of maintenance

✧ Maintenance to repair software faults

- Changing a system to correct deficiencies in the way meets its requirements.

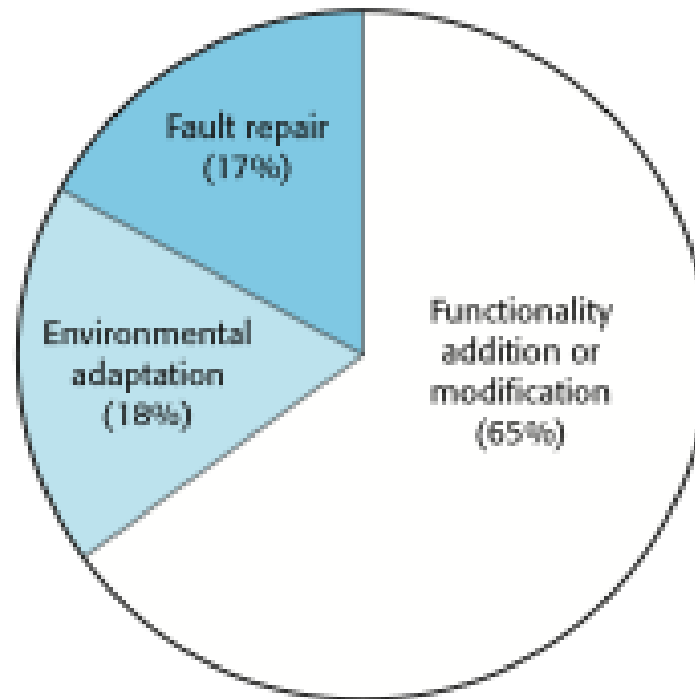
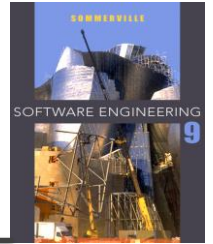
✧ Maintenance to adapt software to a different operating environment

- Changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation.

✧ Maintenance to add to or modify the system's functionality

- Modifying the system to satisfy new requirements.

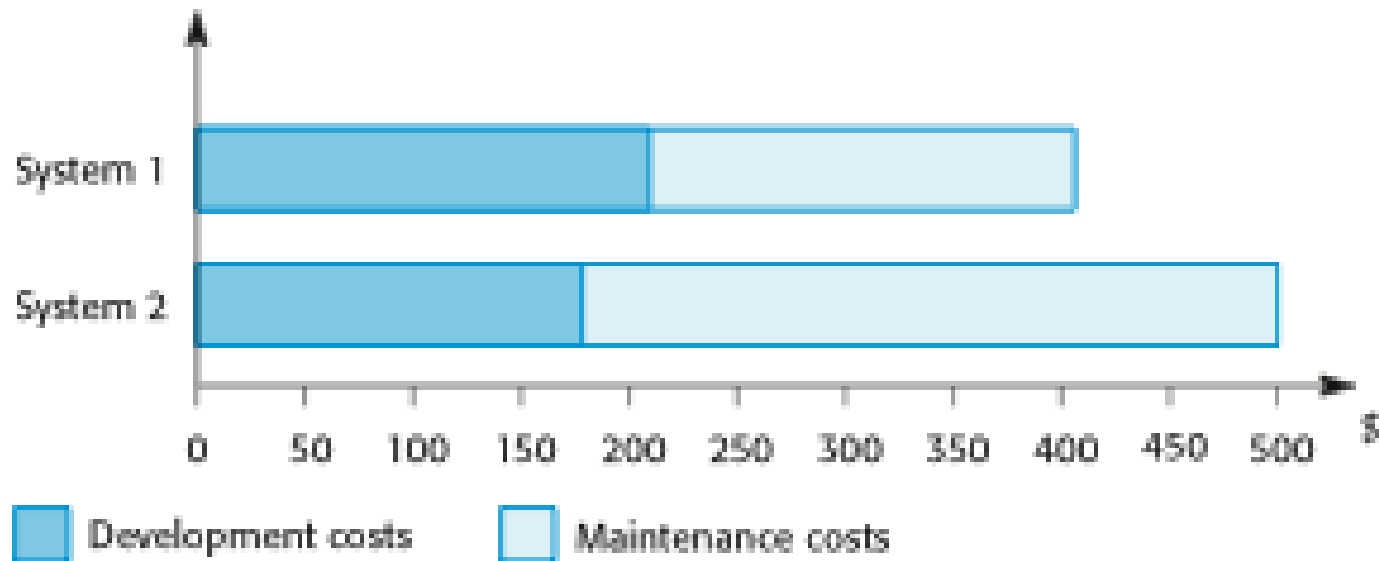
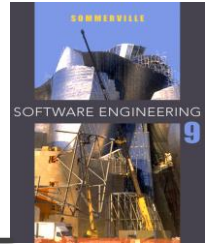
Figure 9.8 Maintenance effort distribution



Maintenance costs

- ✧ Usually greater than development costs (2* to 100* depending on the application).
- ✧ Affected by both technical and non-technical factors.
- ✧ Increases as software is maintained.
Maintenance corrupts the software structure so makes further maintenance more difficult.
- ✧ Ageing software can have high support costs (e.g. old languages, compilers etc.).

Figure 9.9 Development and maintenance costs



Maintenance cost factors

✧ Team stability

- Maintenance costs are reduced if the same staff are involved with them for some time.

✧ Contractual responsibility

- The developers of a system may have no contractual responsibility for maintenance so there is no incentive to design for future change.

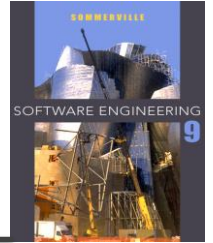
✧ Staff skills

- Maintenance staff are often inexperienced and have limited domain knowledge.

✧ Program age and structure

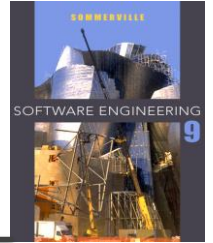
- As programs age, their structure is degraded and they become harder to understand and change.

System re-engineering



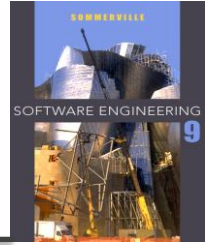
- ✧ Re-structuring or re-writing part or all of a legacy system without changing its functionality.
- ✧ Applicable where some but not all sub-systems of a larger system require frequent maintenance.
- ✧ Re-engineering involves adding effort to make them easier to maintain. The system may be re-structured and re-documented.

Activities Involved In The Reengineering Process:



- ✧ To make legacy software systems easier to maintain, you can reengineer these systems to improve their structure and understandability
- ✧ The activities in reengineering process are:
 1. **Source code translation** Using a translation tool, you can convert the program from an old programming language to a more modern version of the same language or to a different language.
 2. **Reverse engineering** The program is analyzed and information extracted from it. This helps to document its organization and functionality. Again, this process is usually completely automated.

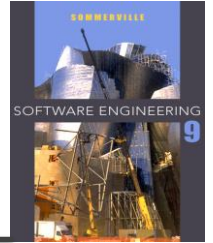
Activities Involved In The Reengineering Process:



3. Program structure improvement The control structure of the program is analyzed and modified to make it easier to read and understand. This can be partially automated, but some manual intervention is usually required.

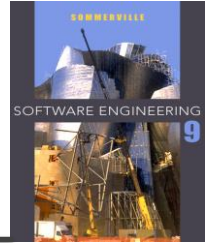
4. Program modularization Related parts of the program are grouped together, and, where appropriate, redundancy is removed. In some cases, this stage may involve architectural refactoring (e.g., a system that uses several different data stores may be refactored to use a single repository). This is a manual process.

Activities Involved In The Reengineering Process:



5. Data reengineering The data processed by the program is changed to reflect program changes. This may mean redefining database schemas and converting existing databases to the new structure. You should usually also clean up the data. This involves finding and correcting mistakes, removing duplicate records, and so on. This can be a very expensive and prolonged process.

Advantages of reengineering



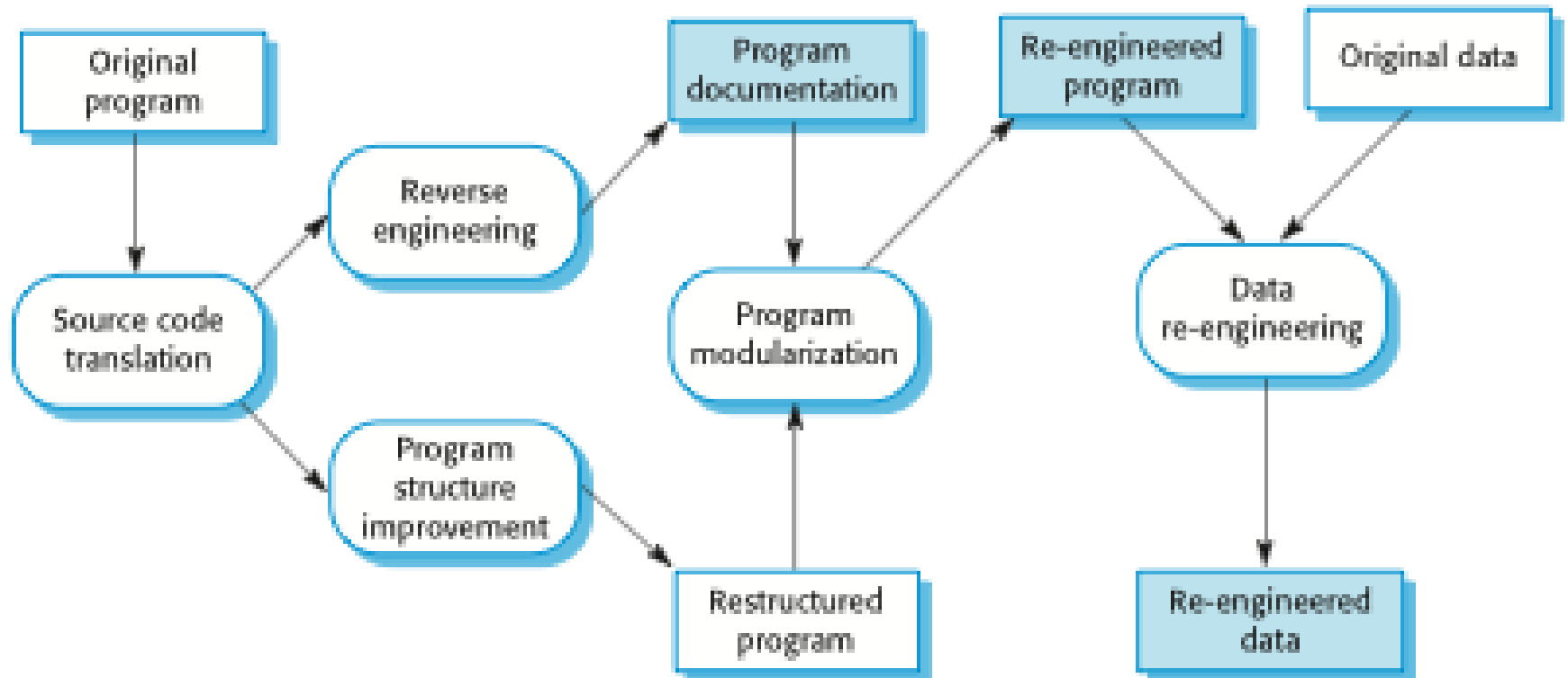
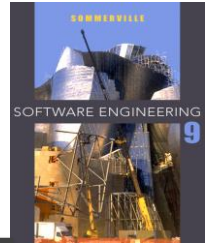
✧ Reduced risk

- There is a high risk in new software development. There may be development problems, staffing problems and specification problems.

✧ Reduced cost

- The cost of re-engineering is often significantly less than the costs of developing new software.

The reengineering process



Reengineering process activities

- ✧ Source code translation
 - Convert code to a new language.
- ✧ Reverse engineering
 - Analyse the program to understand it;
- ✧ Program structure improvement
 - Restructure automatically for understandability;
- ✧ Program modularisation
 - Reorganise the program structure;
- ✧ Data reengineering
 - Clean-up and restructure system data.